Государственное бюджетное общеобразовательное учреждение
«Челябинский областной многопрофильный лицей-интернат для одарённых детей»

Проектная работа

«Разработка веб-платформы для автоматизации проверки знаний: от школьных заданий до олимпиад»

по информатике

Выполнил ученик 8 «Б» класса Кузмичев Кирилл Павлович
Руководитель Арабаджи Андрей Александрович

Челябинск, 2026 год

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ ... 3
ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И ПОСТАНОВКА ЗАДАЧИ ... 5
1.1. Предметная область и назначение платформы ... 5
1.2. Анализ существующего подхода к проверке знаний ... 5
1.3. Аналоги и выбранный подход ... 5
1.4. Требования к разрабатываемому решению ... 6
1.5. Статус MVP и границы проекта ... 7
ВЫВОД ПО ТЕОРЕТИЧЕСКОЙ ГЛАВЕ ... 8
ГЛАВА 2. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ ПРОЕКТА ... 9
2.1. Общая характеристика разрабатываемого продукта ... 9
2.2. Структура исходного кода ... 9
2.3. Архитектура решения ... 10
2.4. Реализованный функционал ... 11
2.5. Основные компоненты и особенности реализации ... 11
2.6. Модель данных ... 13
2.7. Используемые технологии ... 13
2.8. Оценка текущего состояния проекта ... 14
2.9. Личный вклад и этапы работы ... 14
ВЫВОД ПО ПРАКТИЧЕСКОЙ ГЛАВЕ ... 16
ЗАКЛЮЧЕНИЕ ... 17
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ ... 18
ПРИЛОЖЕНИЯ ... 19
Приложение А. Визуальные материалы проекта ... 19
Приложение Б. Схема архитектуры проекта ... 23
Приложение В. Структура базы данных ... 24

ВВЕДЕНИЕ

Тема настоящей проектной работы: «Разработка веб-платформы для автоматизации проверки знаний: от школьных заданий до олимпиад». Работа посвящена созданию и описанию программного прототипа веб-платформы Qubite, предназначенной для автоматизации проверки знаний в разных учебных сценариях: от обычных школьных заданий до олимпиадных и турнирных форматов.

Актуальность. Проверка знаний в школе часто строится на ручной проверке, отдельных формах, таблицах и переписках. Такой подход работает для небольших заданий, но становится неудобным, когда нужно провести турнир, быстро собрать ответы, посчитать результаты, показать рейтинг и сохранить историю участия. Поэтому актуальна разработка единой цифровой платформы, которая объединяет задания, участников, проверку ответов и аналитику.

Проблема. Учителю или организатору приходится тратить время не только на содержание заданий, но и на техническую организацию процесса: регистрацию участников, допуск к заданиям, подсчёт баллов, обработку результатов и объяснение рейтинга. Для учеников проблема заключается в том, что результаты не всегда видны сразу, а участие в олимпиадном формате часто требует отдельных сложных сервисов.

Идея проекта. Предлагается веб-платформа, в которой основным объектом является турнир. Вокруг турнира строятся регистрация, выполнение заданий, проверка ответов, рейтинг, командное участие, аналитика и служебное управление. Такой подход позволяет рассматривать проект не как отдельную форму с вопросами, а как школьный веб-сервис.

Объект исследования — цифровые средства организации и проверки учебных заданий.

Предмет исследования — архитектура, функциональные возможности и способы практической реализации веб-платформы для автоматизации проверки знаний.

Гипотеза. Если создать единую веб-платформу, объединяющую банк заданий, роли участников и организаторов, автоматическую проверку типовых ответов и аналитику результатов, то проведение школьных заданий и олимпиад станет более удобным, быстрым и прозрачным.

Цель проекта — разработать и описать рабочий прототип веб-платформы для автоматизации проверки знаний, который может применяться в школьных заданиях, тренировках и олимпиадных соревнованиях.

Постановка цели по SMART. Для более точной формулировки цели использовался подход SMART:

S	Цель конкретна: разработать прототип веб-платформы с понятным набором функций, связанных с проверкой знаний.
M	Результат измерим: в проекте должны присутствовать роли пользователей, турниры, задачи, автоматическая проверка ответов, рейтинг, база данных и описание архитектуры.

A	Цель достижима, так как реализуется на доступном для школьного проекта стеке: HTML, CSS, JavaScript, Node.js и SQLite.
R	Цель значима для школы, поскольку направлена на упрощение проверки знаний и организации соревнований.
T	Срок выполнения проекта — 2025-2026 учебный год.

Таблица 1 — Постановка цели по критериям SMART

Задачи проекта.

- изучить предметную область цифровой проверки знаний и выделить основные проблемы существующих подходов;
- определить требования к платформе для участников, организаторов и служебных ролей;
- спроектировать структуру web-приложения, включающую клиентскую часть, серверную часть и базу данных;
- реализовать рабочий прототип с турнирами, банком задач, авторизацией, рейтингом и аналитикой;
- описать меры безопасности, ограничения текущей версии и направления дальнейшего развития.

Методы работы.

- анализ литературы, нормативных требований и аналогов;
- декомпозиция системы на функциональные модули;
- проектирование структуры данных;
- прототипирование интерфейса и серверной логики;
- функциональный анализ исходного кода и пользовательских сценариев.

Практическая значимость. Проект может использоваться как основа цифрового инструмента для тренировочных заданий, школьных турниров и олимпиад. Участники получают понятный web-интерфейс, а организаторы — инструменты управления заданиями, участниками и результатами.

Тип проекта и продукт. Тип проекта можно определить как прикладной, практико-ориентированный проект с элементами конструкторской разработки. Продуктом проекта является программный MVP-прототип веб-платформы Qubite. Работа выполнялась в течение одного учебного года — 2025-2026 учебный год.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И ПОСТАНОВКА ЗАДАЧИ

1.1. Предметная область и назначение платформы

Цифровая проверка знаний — это область, в которой информационные технологии используются для подготовки заданий, выдачи их учащимся, сбора ответов, автоматической или полуавтоматической проверки и анализа результатов. Такие системы особенно востребованы в школе, потому что позволяют соединить обучение, контроль знаний и элементы соревнования.

Qubite является рабочим MVP-прототипом платформы для учебных соревнований. Главный объект системы — турнир, вокруг которого строятся регистрация, список допуска, выполнение заданий, рейтинг, лидерборд и аналитика.

С практической точки зрения такая платформа нужна для решения нескольких задач:

- объединения заданий, пользователей и результатов в одной системе;
- снижения доли ручной проверки и ручного подсчёта баллов;
- упрощения организации школьных турниров, тренировок и олимпиад;
- накопления статистики по успеваемости и активности участников;
- создания единой цифровой среды для ученика, организатора и администратора.

Предметная область проекта относится к образовательным web-сервисам, сочетающим функции тестирования, соревнований, управления пользователями и анализа результатов. При этом проект не сводится к обычной форме с вопросами: в нём есть роли, турнирная логика, рейтинговая модель и инфраструктурная подготовка к запуску на сервере.

1.2. Анализ существующего подхода к проверке знаний

В школьной практике обычно используются несколько отдельных инструментов: бумажные задания, электронные формы, таблицы с результатами, мессенджеры и презентации. Такой подход создаёт ряд проблем:

- данные о заданиях и результатах хранятся разрозненно;
- сложно быстро организовать честное соревнование с таймером и рейтингом;
- преподавателю приходится вручную переносить результаты и проверять часть ответов;
- отсутствует единый интерфейс для командного участия и учёта статистики;
- трудно масштабировать процесс на большое количество участников.

Таким образом, актуальной становится задача создания единой платформы, которая объединяет создание заданий, их публикацию, участие учеников и анализ результатов.

Проект решает не абстрактную программистскую задачу, а понятную образовательную проблему: как сделать проверку знаний быстрее, прозрачнее и интереснее для участников.

1.3. Аналоги и выбранный подход

Существуют разные цифровые инструменты для проверки знаний: онлайн-тесты, формы, сервисы интерактивных викторин, электронные таблицы и образовательные платформы. Их можно условно разделить на несколько групп:

Тип инструмента	Преимущества и ограничения
Онлайн-формы	Просты в использовании, но слабо подходят для сложных турниров, ролей, рейтингов и командного участия.
Сервисы викторин	Хорошо вовлекают участников, но часто ограничены форматами вопросов и правилами платформы.
Электронные таблицы	Удобны для подсчётов, но не дают полноценного пользовательского интерфейса и автоматического runtime-сценария.
Олимпиадные системы	Подходят для серьёзных соревнований, но обычно сложнее для школьного применения и требуют отдельной настройки.

Таблица 2 — Сравнение типов инструментов проверки знаний

В проекте Qubite выбран гибридный подход: система должна быть простой для школьного применения, но при этом включать важные элементы веб-платформы — роли, серверную часть, базу данных, рейтинги, списки допуска и защиту пользовательских действий.

1.4. Требования к разрабатываемому решению

На основе анализа предметной области и практических потребностей платформы можно сформулировать следующие требования:

Группа требований	Содержание
Функциональные	Регистрация и вход, роли пользователей, создание турниров, списки допуска, добавление задач, участие в соревнованиях, рейтинг, лидерборд и аналитика.
Интерфейсные	Понятный web-интерфейс, пригодный для ученика, организатора и администратора.
Технические	Работа в браузере, клиент-серверная архитектура, хранение данных в SQLite, возможность локального запуска и размещения на сервере.
Безопасность	Защита авторизации, контроль сессий, CAPTCHA/anti-bot слой, ограничение подозрительных запросов, аудит чувствительных действий.
Практические	Возможность использования как для обычных школьных заданий, так и для олимпиадных и турнирных сценариев.

Таблица 3 — Требования к разрабатываемому решению

Дополнительно важно, чтобы проект был понятен не только разработчику, но и пользователю без специальной технической подготовки. Поэтому в записке отдельно описаны функции, роли и структура системы.

1.5. Статус MVP и границы проекта

Понятие MVP означает минимально жизнеспособный продукт: версию системы, в которой уже есть основные функции, но часть улучшений ещё остаётся в плане развития. Для Qubite такой статус подходит лучше всего.

Границы текущей версии. В проекте уже реализованы турниры, роли, базовая проверка задач, рейтинг, импорт из Excel, администрирование, Telegram-бот управления и настройки для размещения на сервере. Следующие функции пока не реализованы и относятся к дальнейшему развитию: code runner для задач по программированию, полноценная AI-проверка ответов, апелляции и визуальные типы задач.

Чёткое определение границ помогает объективно оценить текущее состояние продукта и спланировать его дальнейшее развитие.

ВЫВОД ПО ТЕОРЕТИЧЕСКОЙ ГЛАВЕ

В теоретической части установлено, что автоматизация проверки знаний является актуальной задачей для школы, особенно если речь идёт не только о простом тестировании, но и о турнирах, рейтинге, командном участии и анализе результатов. Рассмотренные подходы показывают, что отдельные формы, таблицы и викторины решают только часть проблемы, поэтому для проекта выбран формат единой веб-платформы.

Итогом теоретической главы стала постановка требований к решению: система должна быть понятной для ученика, полезной для организатора, иметь серверную часть, базу данных, роли пользователей, автоматическую проверку типовых ответов и честно обозначенные границы MVP. Эти требования используются в практической части при описании реализации проекта Qubite.

ГЛАВА 2. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ ПРОЕКТА

2.1. Общая характеристика разрабатываемого продукта

Проект Qubite представляет собой рабочий программный прототип, близкий к формату MVP. В нём уже реализованы основные пользовательские сценарии, но отдельные части интерфейса и логики ещё требуют доработки.

Разработка выполнена как монолитное web-приложение с клиентской и серверной частями. Пользователь работает через браузер, а сервер принимает запросы, обрабатывает их и взаимодействует с базой данных. Такой вариант архитектуры удобен для школьного проекта, потому что в одной работе можно рассмотреть интерфейс, серверную логику, хранение данных, безопасность и подготовку к размещению на сервере.

На верхнем уровне репозиторий разделён на несколько смысловых частей:

- `front/` — клиентская часть;
- `back/` — серверная часть;
- `deploy/` — файлы для развертывания и настройки безопасности;
- `docs/` — рабочая документация по архитектуре, ролям, разработке и размещению проекта на сервере;
- корневые HTML-страницы и сопроводительные документы проекта.

Клиентская часть реализована на HTML, CSS и JavaScript без использования фреймворков, а серверная — на Node.js, Express и SQLite. Такой подход делает проект более понятным и самодостаточным.

2.2. Структура исходного кода

Содержательно структура кода может быть представлена так:

Каталог или файл	**Назначение**
`front/js/app.js`	Основная логика интерфейса, управление состоянием экранов, форм и пользовательских сценариев.
`front/js/api.js`	Связь клиентской части с REST API и локальное хранение состояния.
`front/css/styles.css`	Оформление интерфейса, темы, компоненты и визуальная система проекта.
`back/server.js`	Точка входа серверного приложения и описание API-маршрутов.
`back/src/db.js`	Схема базы данных, бизнес-операции и логика работы с сущностями.
`back/src/task-runtime.js`	Автоматическая проверка ответов и подготовка конфигурации задач.

`back/src/security.js`	Хеширование паролей, генерация токенов и вспомогательные функции безопасности.
`back/src/imports.js`	Импорт задач и списков участников из Excel-файлов.
`back/src/telegram-bot.js`	Запуск Telegram-бота для owner и модераторов.
`back/src/telegram/`	Обработчики Telegram-команд, меню, права доступа и уведомления.
`deploy/`	Подготовка к публикации проекта и настройка серверного окружения.

Таблица 4 — Структура исходного кода проекта

Отдельно стоит отметить наличие набора `.md`-документов. Они фиксируют архитектуру, роли, разработку, deploy, безопасность, технический долг и roadmap. Это помогает поддерживать проект в порядке и упрощает дальнейшую доработку.

2.3. Архитектура решения

Архитектура проекта является клиент-серверной. Её можно описать следующим образом:

- клиентская часть отображает интерфейс и отправляет запросы;
- сервер на Express обрабатывает маршруты авторизации, турниров, команд, задач, поддержки и аналитики;
- база данных SQLite хранит пользователей, турниры, задачи, команды, сессии, результаты, историю рейтинга и журнал аудита;
- внешние сервисы используются для CAPTCHA, OAuth-авторизации, email-уведомлений и Telegram-интеграции;
- размещение на сервере предполагает Nginx, PM2, Cloudflare и firewall-настройки.

Схематически работу приложения можно представить так:

Уровень	**Что происходит**
Браузер	Пользователь открывает страницу, входит в аккаунт, выбирает турнир, решает задачи и смотрит рейтинг.
Клиентский JavaScript	Отправляет запросы к API, обновляет интерфейс, хранит часть состояния на стороне браузера.
Express-сервер	Проверяет права, обрабатывает данные, запускает бизнес-логику и отдаёт ответы клиенту.
SQLite	Сохраняет аккаунты, турниры, задачи, попытки, команды, рейтинг, настройки и аудит.
Инфраструктура	Nginx и Cloudflare принимают внешний трафик, PM2 удерживает Node-процесс, firewall ограничивает доступ.

Таблица 5 — Уровни работы приложения

2.4. Реализованный функционал

На текущем этапе в проекте уже реализованы следующие возможности:

- регистрация, вход, восстановление пароля, email-подтверждение и управление сессиями;
- роли `user`, `organizer`, `moderator`, `admin`, `owner`;
- создание и редактирование турниров;
- создание банка задач;
- автоматическая проверка ответов типов `single_choice`, `multiple_choice`, `short_text`, `number`;
- индивидуальное и командное участие;
- режимы допуска к турнирам: открытый вход, регистрация, список допуска, код;
- лидерборд, пользовательская статистика и история изменения рейтинга;
- импорт задач и списков участников из Excel;
- helper-коды для отдельных экранов и помощников;
- административная, модераторская и owner-части;
- Telegram-бот для управления платформой;
- страница поддержки и legal-страницы для пользователей.

Отдельно стоит отметить функцию ежедневных заданий: платформа умеет автоматически формировать специальный ежедневный турнир из банка задач.

Текущую реализацию можно разделить по ролям:

Роль	**Основные возможности**
`user`	Регистрация, вход, участие в турнирах, решение задач, рейтинг, аналитика, команда.
`organizer`	Создание турниров, работа с задачами, импорт участников, просмотр результатов.
`moderator`	Проверка задач, заявок организаторов и часть операций контроля пользователей.
`admin`	Управление пользователями, турнирами, задачами, командами, аналитикой и аудитом.
`owner`	Максимально защищённая роль: системные настройки, режим обслуживания, hard delete и восстановление пользователей.

Таблица 6 — Реализованные возможности по ролям

Почему это важно для проекта. Наличие ролей важно для практического применения платформы: ученики участвуют, организаторы создают события, модераторы контролируют качество, администраторы управляют системой.

2.5. Основные компоненты и особенности реализации

В серверной части проекта выделяются несколько ключевых компонентов:

Компонент	Роль в системе
Модуль конфигурации	Загружает параметры среды, адреса сервиса, настройки базы данных, CAPTCHA и OAuth.
HTTP-сервер	Принимает запросы и распределяет их по API-маршрутам.
Подсистема БД	Хранит данные и реализует прикладные операции над ними.
Подсистема безопасности	Отвечает за хеширование паролей, токены, ограничения запросов и защиту сессий.
Подсистема задач	Готовит задания к использованию и проверяет пользовательские ответы.
Подсистема импорта	Позволяет загружать задачи и участников из Excel-файлов.
Подсистема рейтинга	Фиксирует изменения рейтинга и объясняет пользователю, почему он изменился.
Подсистема поддержки	Позволяет пользователям отправлять обращения и получать ответы в режиме live-обновлений.
Telegram-бот	Даёт owner и модераторам быстрый доступ к настройкам, аналитике и служебным действиям.
Клиентский API-слой	Скрывает детали запросов и синхронизирует данные на стороне интерфейса.

Таблица 7 — Основные компоненты серверной части

Существенной особенностью проекта является то, что большая часть бизнес-логики сосредоточена в модуле `back/src/db.js`. Это удобно для небольшого проекта, так как весь прикладной функционал находится рядом и его проще анализировать. Однако при дальнейшем развитии платформы такой модуль желательно разделить на более узкие сервисы.

При разработке проекта особое внимание уделено вопросам безопасности. В проекте реализованы следующие меры:

- хеширование паролей через `scrypt`;
- управление сессиями пользователей;
- email-подтверждение и challenge-flow для восстановления пароля;
- CAPTCHA для критичных форм на базе Cloudflare Turnstile;
- ограничение частоты однотипных запросов;
- аудит важных действий;
- защита owner-роли от обычного понижения и блокировки;
- режим обслуживания с секретным обходом;
- отдельные конфигурации для reverse proxy и сетевой защиты.

За счёт этих мер прототип становится надёжнее и ближе к реальному веб-сервису.

2.6. Модель данных

Основное хранилище проекта — SQLite. Для MVP это практичный выбор: база хранится в одном файле, легко запускается локально и не требует отдельного серверного процесса. В проекте включены внешние ключи, WAL-журнал и индексы для часто используемых запросов.

Основные группы таблиц:

Группа данных	**Примеры таблиц и назначение**
Пользователи и доступ	`users`, `sessions`, `auth_challenges`, `password_reset_tickets`, `oauth_states`.
Турниры и задания	`tournaments`, `task_bank`, `tournament_tasks`, `tournament_entries`, `tournament_submissions`.
Команды и списки допуска	`teams`, `team_members`, `tournament_roster_entries`, `tournament_helper_codes`.
Рейтинг и аналитика	`rating_changes`, `system_stats_history`, `site_visits`, `email_logs`.
Управление и безопасность	`audit_log`, `system_settings`, `blocked_emails`, `ip_blocklist`, `telegram_access`.
Поддержка	`support_chats`, `support_messages`.

Таблица 8 — Основные группы таблиц базы данных

Такая модель данных отражает полный жизненный цикл платформы: вход, участие, проверку, рейтинг, поддержку, модерацию и администрирование.

2.7. Используемые технологии

С технической точки зрения проект построен на практичном и понятном для учебной работы стеке:

Технология	**Назначение**
HTML5	Формирование структуры web-страниц и точек входа интерфейса.
CSS3	Оформление приложения, темы, адаптация интерфейса и визуальные компоненты.
JavaScript	Клиентская логика интерфейса и серверная прикладная логика.
Node.js	Среда выполнения серверной части.
Express	Организация REST API и маршрутов приложения.
SQLite	Локальное хранение данных проекта.

`xlsx`	Импорт данных из электронных таблиц Excel.
Cloudflare Turnstile	Базовая защита форм от автоматических запросов.
Google, Yandex, VK ID, Telegram OAuth	Альтернативные способы входа в систему.
Telegram Bot API	Служебное управление платформой через Telegram.
Nginx, PM2, Cloudflare	Подготовка проекта к размещению на сервере.

Таблица 9 — Используемые технологии и их назначение

Выбор такого стека подходит для школьного проекта: он позволяет реализовать настоящую клиент-серверную архитектуру и при этом не требует чрезмерно сложной инфраструктуры.

2.8. Оценка текущего состояния проекта

Текущее состояние Qubite можно оценить как рабочий программный прототип:

- существует серверная логика и база данных;
- есть пользовательский интерфейс;
- реализованы несколько пользовательских ролей;
- доступны прикладные сценарии создания турниров и решения задач;
- предусмотрены меры безопасности, поддержка Telegram-управления и подготовка к размещению на сервере.

Одновременно проект пока нельзя считать полностью завершённой промышленной системой. В текущей версии интерфейсы организатора, модератора и администратора слабее основной части для участника, режим проведения турнира требует стабилизации, а code runner, AI-проверка, апелляции и визуальные типы задач относятся к roadmap.

Итоговая оценка состояния. Наиболее точная формулировка текущего состояния проекта — рабочий функциональный MVP-прототип. Основные сценарии уже реализованы, но часть функций ещё требует развития и проверки.

2.9. Личный вклад и этапы работы

Проект выполнен автором самостоятельно. В рамках работы были изучены требования к школьному индивидуальному проекту, выбрана тема, определена идея продукта, продуманы основные пользовательские сценарии, спроектирован и реализован интерфейс, написана серверная часть и подготовлена документация.

Работу над проектом можно условно разделить на несколько этапов:

- выбор темы и формулировка проблемы;
- анализ аналогов и требований к платформе;
- проектирование интерфейса и пользовательских сценариев;
- разработка и настройка клиентской и серверной частей;
- проверка работы основных сценариев: вход, турниры, задачи, рейтинг, профиль;

- подготовка пояснительной записки и материалов для защиты.

Наиболее сложной частью проекта стал дизайн интерфейса. При разработке важно было не просто разместить кнопки и формы, а сделать систему понятной: чтобы участник быстро находил турниры и результаты, а интерфейс воспринимался как законченная часть проекта. Поэтому особое внимание уделено главной странице, визуальному стилю, карточкам, аналитическим блокам и экрану профиля.

ВЫВОД ПО ПРАКТИЧЕСКОЙ ГЛАВЕ

В практической части описано, что Qubite реализован как рабочий MVP-прототип веб-платформы для учебных соревнований. Проект включает клиентский интерфейс, сервер на Node.js и Express, базу данных SQLite, роли пользователей, турниры, банк задач, автоматическую проверку части ответов, рейтинг, аналитику, импорт данных, Telegram-бота и элементы подготовки к размещению на сервере.

Главный результат практической части состоит в том, что создана не отдельная демонстрационная страница, а система с несколькими связанными модулями. На защите целесообразно продемонстрировать главный экран, раздел турниров, решение задания, профиль с аналитикой и рейтинг. При этом текущая версия честно описывается как MVP: она работает в основных сценариях, но сохраняет направления для развития.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы была рассмотрена тема разработки веб-платформы для автоматизации проверки знаний. Разработанный проект Qubite содержит важные элементы прикладного продукта: клиентский интерфейс, серверную часть, базу данных, роли пользователей, банк задач, механизм проведения турниров, автоматическую проверку части ответов и аналитические возможности.

Поставленная цель — разработать и описать рабочий прототип платформы для автоматизации проверки знаний — в целом достигнута. Рассмотрены предметная область, требования к системе, структура проекта, архитектура решения, ключевой функционал, безопасность и ограничения текущей версии. Сформулированная в начале работы гипотеза подтверждается: единая web-платформа действительно способна упростить организацию проверки знаний и соревнований, потому что объединяет задания, участников, результаты, рейтинг и администрирование в одной системе.

В ходе работы были решены следующие задачи:

- изучена предметная область цифровой проверки знаний и выделены основные проблемы существующих подходов;
- определены требования к платформе для участников, организаторов и служебных ролей;
- спроектирована структура web-приложения, включающая клиентскую часть, серверную часть и базу данных;
- реализован рабочий прототип с турнирами, банком задач, авторизацией, рейтингом и аналитикой;
- описаны меры безопасности, ограничения текущей версии и направления дальнейшего развития.

Практическая ценность проекта состоит в том, что его можно развивать как основу цифрового инструмента для школы: для домашних заданий, тренировочных работ, школьных турниров и олимпиад. Сильной стороной работы является то, что проект включает не только внешний интерфейс, но и серверную часть, хранение данных, роли, безопасность, рейтинг, импорт и документацию.

Дальнейшее развитие проекта целесообразно вести по нескольким направлениям: улучшить интерфейсы организатора, модератора и администратора, стабилизировать режим проведения турнира, усилить защиту от автоматических запросов, добавить апелляции, задачи по программированию с безопасным runner/sandbox и более сложные типы заданий.

В результате работы получился содержательный и практически значимый программный прототип, который можно развивать как основу школьной платформы для автоматизации проверки знаний.

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. Положение об индивидуальном проекте обучающихся ООО ГБОУ «Челябинского областного многопрофильного лицея-интерната для одарённых детей». – PDF-документ, предоставленный пользователем.
2. ГОСТ Р 2.106-2019. Единая система конструкторской документации. Текстовые документы. – PDF-документ, предоставленный пользователем.
3. Прессман Р. С., Максим Б. Р. Инженерия программного обеспечения: практический подход. – 8-е изд. – М.: Вильямс, 2016. – 976 с.
4. Соммервилл И. Инженерия программного обеспечения. – 10-е изд. – Pearson, 2016. – 816 с.
5. Роббинс Дж. Н. Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics. – 5th ed. – O'Reilly Media, 2018. – 808 с.
6. Duckett J. HTML and CSS: Design and Build Websites. – Indianapolis: Wiley, 2011. – 490 с.
7. Duckett J. JavaScript and JQuery: Interactive Front-End Web Development. – Indianapolis: Wiley, 2014. – 640 с.
8. Гриншкун В. В., Краснова Г. А. Современная цифровая образовательная среда: ресурсы, средства, сервисы: монография. – М.: Проспект, 2021. – 168 с.
9. MDN Web Docs. Документация по HTML, CSS и JavaScript [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/ (дата обращения: 27.04.2026).
10. Node.js Documentation. Официальная документация среды Node.js [Электронный ресурс]. – Режим доступа: https://nodejs.org/api/documentation.html (дата обращения: 27.04.2026).
11. Express Documentation. Routing and basic routing [Электронный ресурс]. – Режим доступа: https://expressjs.com/en/guide/routing.html ; https://expressjs.com/en/starter/basic-routing.html (дата обращения: 27.04.2026).
12. SQLite Documentation. Официальная документация SQLite [Электронный ресурс]. – Режим доступа: https://sqlite.org/docs.html (дата обращения: 27.04.2026).
13. OWASP Cheat Sheet Series. Authentication Cheat Sheet [Электронный ресурс]. – Режим доступа: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (дата обращения: 27.04.2026).
14. Cloudflare Turnstile Documentation [Электронный ресурс]. – Режим доступа: https://developers.cloudflare.com/turnstile/ (дата обращения: 27.04.2026).
15. Yandex Cloud. Getting started with Yandex SmartCaptcha [Электронный ресурс]. – Режим доступа: https://yandex.cloud/en/docs/smartcaptcha/quickstart (дата обращения: 27.04.2026).
16. Atlassian. How to write SMART goals [Электронный ресурс]. – Режим доступа: https://www.atlassian.com/blog/productivity/how-to-write-smart-goals (дата обращения: 27.04.2026).

ПРИЛОЖЕНИЯ

Приложение А. Визуальные материалы проекта

В приложении приведены скриншоты основных экранов проекта. По ним можно увидеть общий стиль платформы, процесс входа, рабочее пространство участника, раздел турниров, решение задачи, профиль с аналитикой и командную работу.

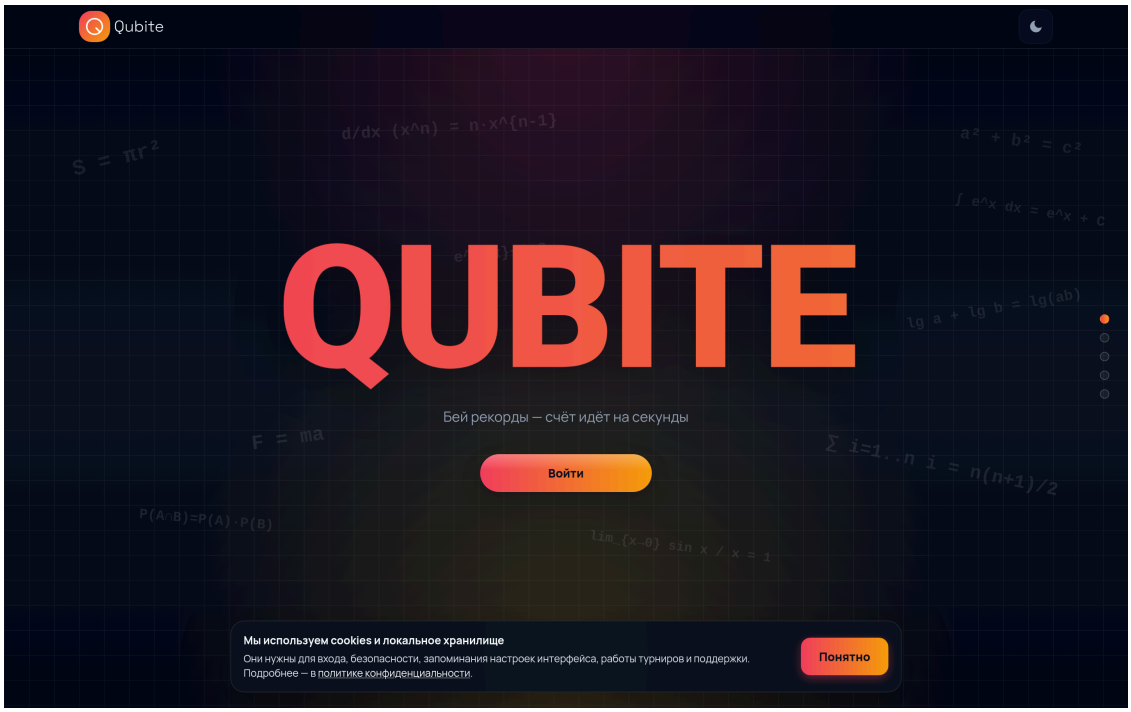


Рисунок 1 — Главная страница платформы Qubite

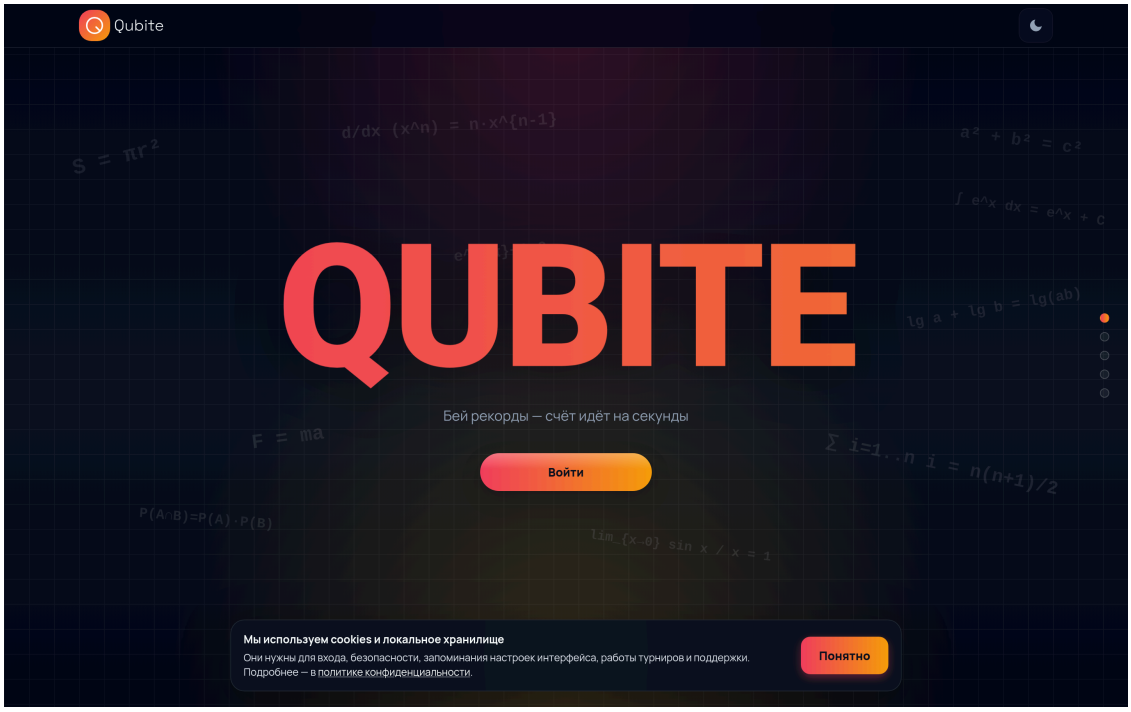


Рисунок 2 — Модалка входа и регистрации

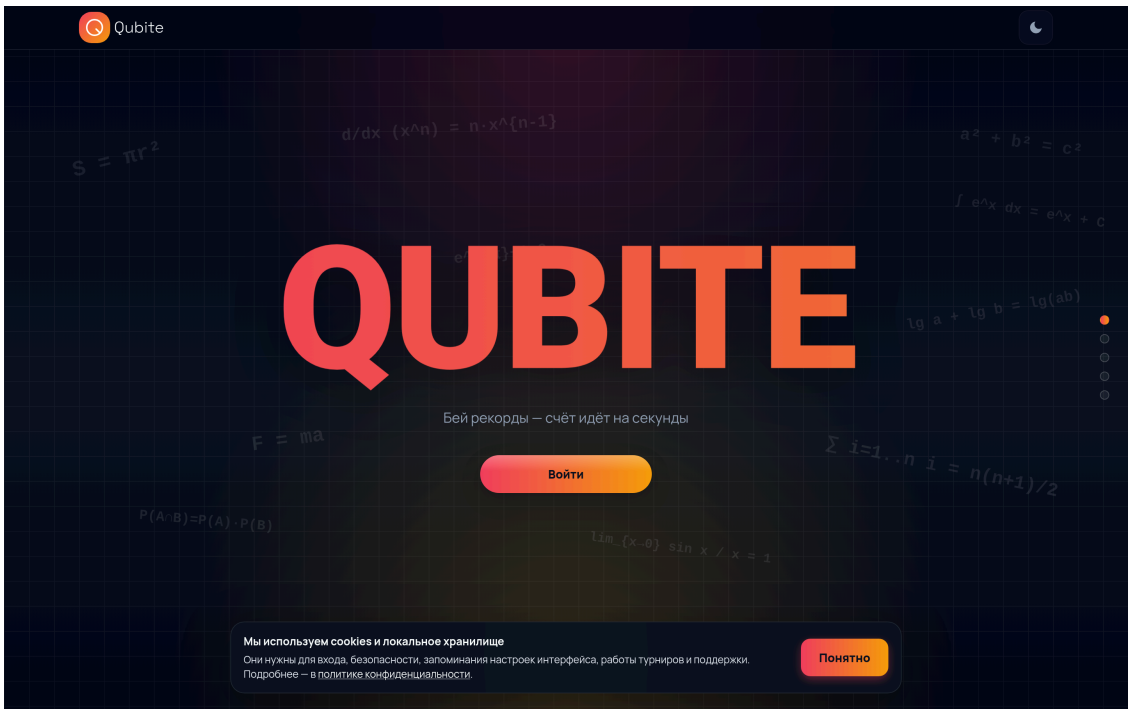


Рисунок 3 — Дашборд участника после входа

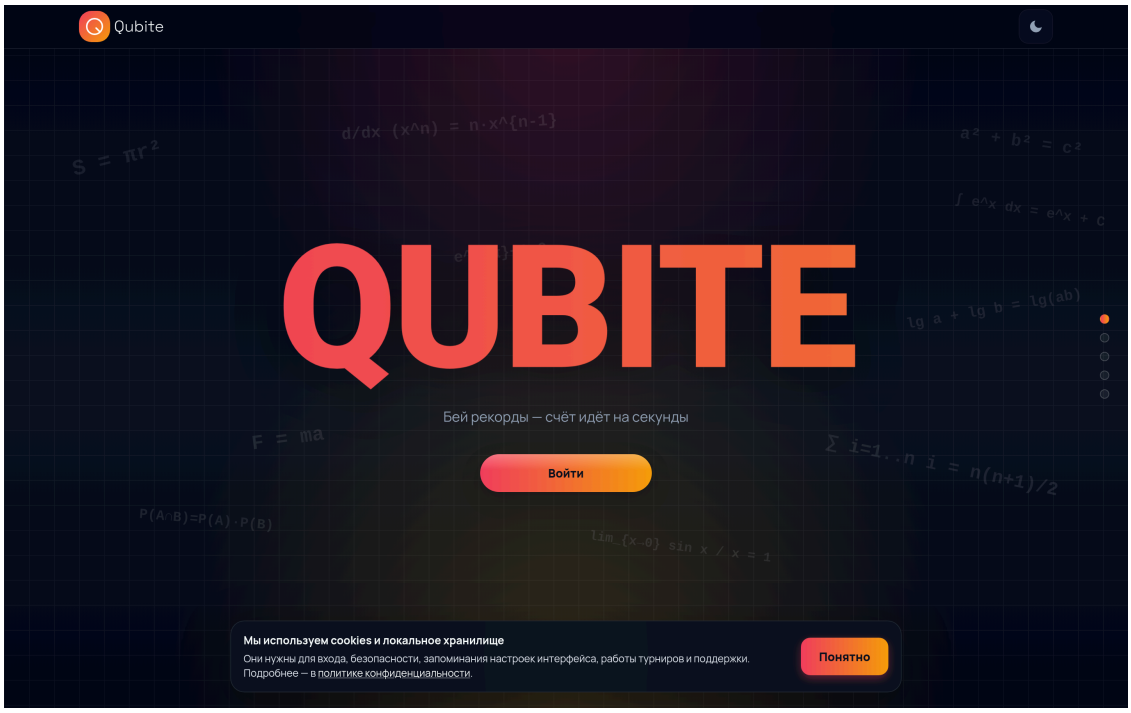


Рисунок 4 — Раздел турниров: список соревнований

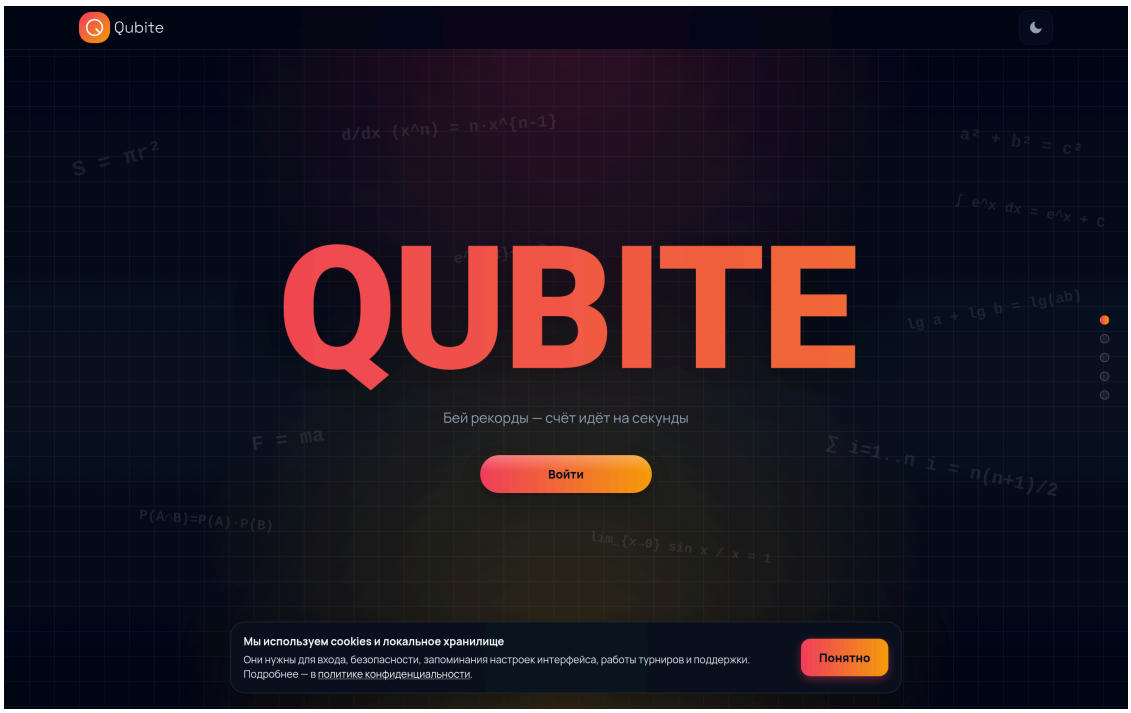


Рисунок 5 — Решение задачи внутри турнира

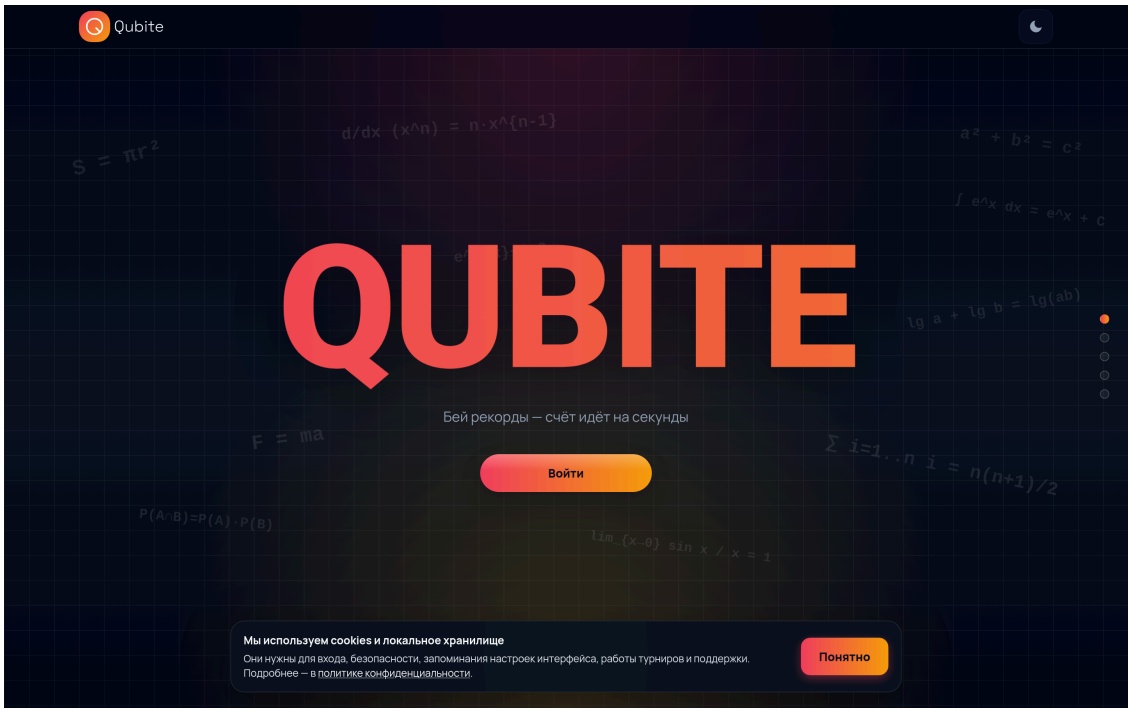


Рисунок 6 — Профиль участника с рейтингом и аналитикой

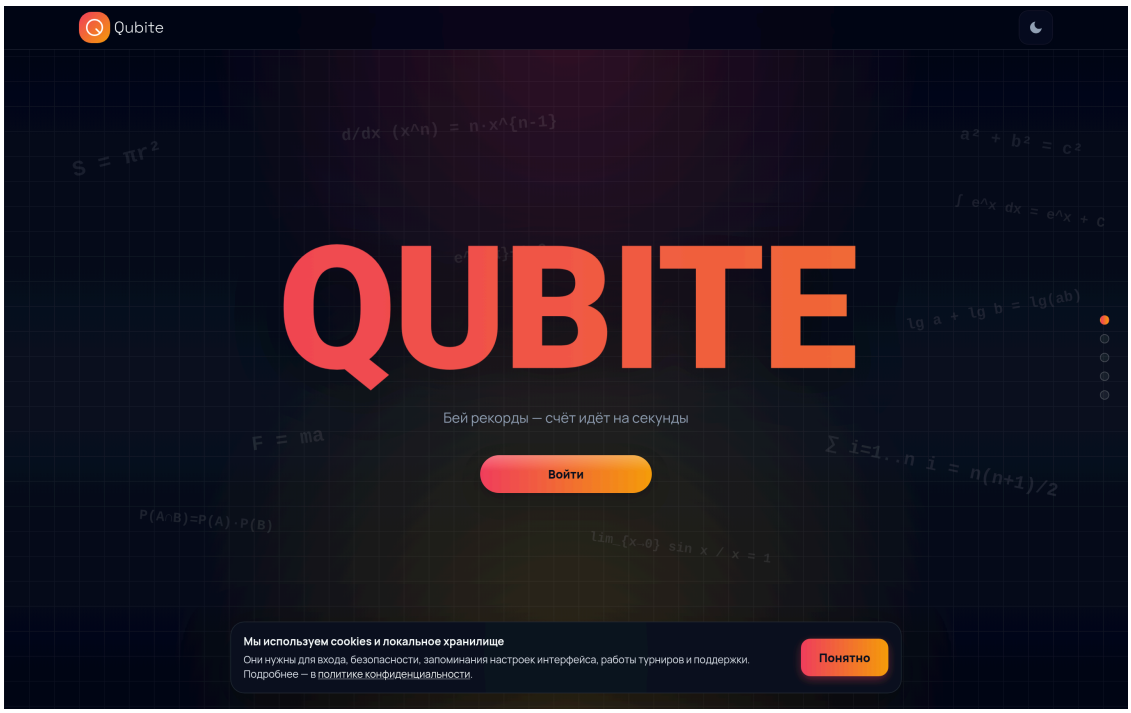
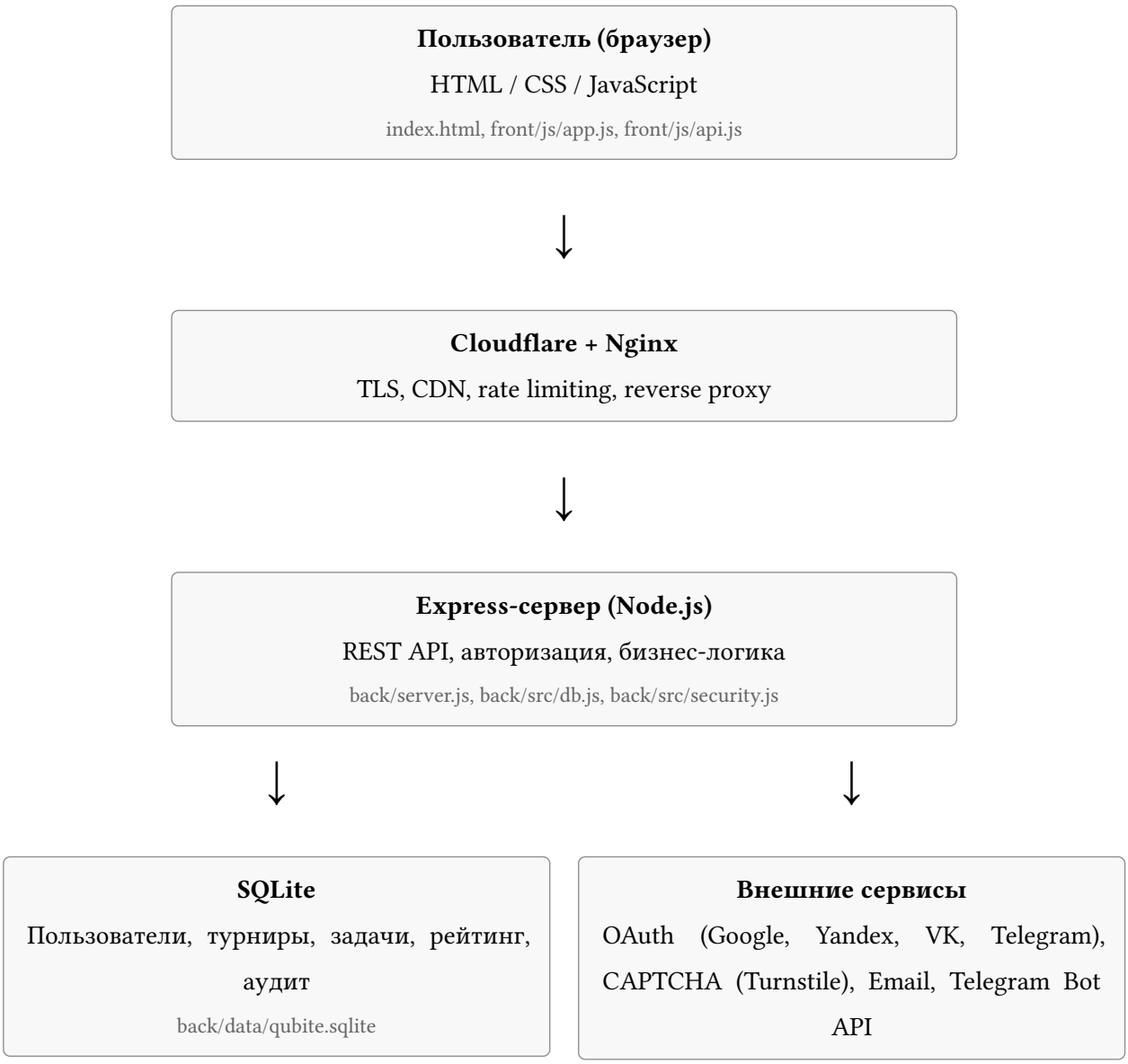


Рисунок 7 — Командная аналитика и работа команды

Приложение Б. Схема архитектуры проекта

Схема отражает основные уровни и связи в архитектуре платформы Qubite.



Приложение В. Структура базы данных

В таблице перечислены все таблицы базы данных проекта, сгруппированные по назначению.

Группа	Таблица	Назначение
Пользователи и доступ	users	Аккаунты, роли, email, статус подтверждения
	sessions	Активные сессии пользователей
	auth_challenges	Challenge-коды для подтверждения email
	password_reset_tickets	Токены восстановления пароля
	oauth_states	Состояния OAuth-потоков (PKCE)
	blocked_emails	Заблокированные email-адреса
	ip_blocklist	Заблокированные IP-адреса
Турниры и задания	tournaments	Турниры: настройки, сроки, режимы допуска
	task_bank	Банк задач с типами и вариантами ответов
	tournament_tasks	Привязка задач к турнирам
	tournament_entries	Записи участников в турнирах
	tournament_submissions	Ответы участников на задачи
	tournament_task_progress	Прогресс решения задач
	tournament_task_drafts	Черновики ответов
	tournament_roster_entries	Списки допуска к турнирам
	tournament_helper_codes	Коды-помощники для экранов турнира
Команды	teams	Команды участников
	team_members	Состав команд
Рейтинг и аналитика	rating_changes	История всех изменений рейтинга
	system_stats_history	Снимки системной статистики
	site_visits	Посещаемость платформы
	email_logs	Журнал отправленных email
Управление	audit_log	Аудит чувствительных действий
	system_settings	Системные настройки (тумблеры owner)

	organizer_applications	Заявки на роль организатора
	telegram_access	Динамические права доступа в Telegram-боте
Поддержка	support_chats	Чаты поддержки пользователей
	support_messages	Сообщения в чатах поддержки

Таблица 10 — Таблицы базы данных проекта Qubite